NEO: A DECENTRALIZED SOCIAL MEDIA PLATFORM

Authors:

Wilayat Ali (F22-BSCS-006)
Hassan Raza (F22-BSCS-022)
Maham Zaka (F22-BSCS-013)

BSCS 7th Semester

Supervisor:
[Dr. Huma Israr]

DEPARTMENT OF INFORMATION ENGINEERING TECHNOLOGY
FACULTY OF COMPUTER SCIENCE
NATIONAL SKILLS UNIVERSITY ISLAMABAD

29 January - 2026

Letter of Undertaking

We certify that project work titled "Neo: A Decentralized Social Media Platform" is our own work. The work has not been presented elsewhere for assessment. Where material has been used from other sources it has been properly acknowledged / referred.

Name: Wilayat Ali
Registration No: F22-BSCS-006
Signature of Student: _____

Name: Hassan Raza
Registration No: F22-BSCS-022
Signature of Student: _____

Name: Maham Zaka
Registration No: F22-BSCS-013
Signature of Student: _____

Abstract

As digital communication becomes a fundamental human right, the reliance on centralized social media platforms introduces significant risks, including mass surveillance, censorship, and service outages in internet-deprived regions. **Neo** is developed to address these issues by providing a fully decentralized, offline-first social networking environment for Android devices.

By eliminating the need for central servers, the application utilizes Bluetooth mesh networking to establish a peer-to-peer (P2P) infrastructure. Information is propagated through the network using an epidemic-style gossip protocol, ensuring eventual consistency even without internet connectivity. To maintain data integrity and authenticity, Neo implements the Ed25519 cryptographic signature scheme, ensuring that every post is verifiable and tamper-proof. The system utilizes a modern Android stack, including Jetpack Compose for the UI and Room for local-first persistent storage. This project serves as a robust solution for resilient communication in emergency scenarios, remote locations, and privacy-sensitive communities.

Keywords: Decentralized, Mesh Network, Peer-to-Peer, Bluetooth, Cryptography, Ed25519.

Acknowledgement

We would like to give our thanks to all those who helped us in completing this project and report. A special thanks to our project supervisor, **Dr. Huma Israr**, who shared her knowledge and suggested different ideas for the project and for writing this report.

We would also like to give our appreciation to the faculty and staff of the Information Engineering Technology Department who supported us throughout this process and allowed us to use the necessary equipment and resources required to complete the project "Neo: A Decentralized Social Media Platform".

A special thanks to the team members, **Hassan Raza** and **Maham Zaka**, who have done their parts and helped each other through every stage of this project. We also appreciate the guidance given by other supervisors and the panel in our project presentation, especially the panel's advice on how to improve our project in the future.

Preface

This report outlines the development of Neo, a serverless, peer-to-peer social networking solution designed to operate in offline environments using Bluetooth mesh technology.

Contents

List of Figures

Chapter 1

Background & Introduction

1.1 Background

The modern digital landscape is dominated by a centralized "hub-and-spoke" model where communication relies entirely on intermediate servers. This architecture creates critical vulnerabilities, including single points of failure and systemic censorship. Neo addresses these by providing a 100% serverless alternative.

1.1.1 Resilience and Accessibility

In internet-deprived regions or during natural disasters, centralized apps become unusable. Neo facilitates communication by allowing data to "hop" between devices via a local-first mesh network.

1.1.2 Cryptographic Trust

In a decentralized environment, trust is established through math rather than authority. By utilizing Ed25519 signatures, Neo ensures that all content is authentic and tamper-proof without requiring a central identity provider.

1.2 Problem Statement

Current digital communication ecosystems suffer from a critical "Centralization Trap". This dependency creates three primary problems that Neo aims to resolve:

1. **Infrastructural Fragility**: Communication is frequently paralyzed by internet outages or disasters, leading to a total loss of community coordination.
2. **Censorship and Surveillance**: Centralized servers allow for top-down control and monitoring, stripping users of their autonomy.
3. **Trust Deficit**: Establishing the authenticity of data in an open P2P network is difficult without a decentralized verification mechanism like Ed25519 signatures.

1.3 Introduction

In the modern digital era, the growing influence of social media platforms has transformed global communication. As more individuals rely on these digital spaces for daily interactions, the reliance on centralized infrastructure has become a significant point of concern. Most current social networking services are governed by central authorities that manage data and connectivity, leading to repercussions such as mass surveillance, data breaches, and the risk of absolute censorship. When conversation and data flow are managed manually or by single entities, the authenticity and privacy of what is shared can be put into question.

While global online systems have emerged to facilitate communication, they often fail in scenarios where internet access is restricted or non-existent. In regions like Pakistan, or in emergency scenarios, the consequences of this centralized reliance include a total loss of communication during network outages or natural disasters. Furthermore, traveling or visiting remote areas without reliable internet makes maintaining social connectivity riskier and more difficult. To solve these issues in a closed or internet-deprived environment, we are designing Neo, a serverless social media application that allows users to broadcast text-based posts and images, interact with peers, and maintain a verifiable digital identity through an offline-first mesh network.

Maintaining land-like digital records—such as post history and identity—on a local-first, decentralized ledger makes the process more transparent for everyone. It empowers users to find and share multimedia information efficiently and safely from their own devices without needing a central middleman.

To build Neo, we have utilized the Kotlin language and the Jetpack Compose framework for a modern, responsive user interface. Initially, a hybrid stack involving Firebase was considered (as seen in our early logs), but as discussed in our project review on December 15, 2025, we shifted entirely to a decentralized model to ensure total server independence. For the back-end data persistence, we are using the Room Persistence Library to manage text and image metadata, and for communication, we utilize Bluetooth Classic/BLE through a custom-built Gossip Protocol. Cryptographic authenticity is the backbone of our system, using the Ed25519 signature scheme to ensure that every interaction—whether text or image—is secure and verifiable. These technologies were chosen because they align with modern trends in Android development and provide the high efficiency required for peer-to-peer mesh networking.

1.3.1 Purpose

The purpose of Neo is to provide a censorship-resistant communication tool that ensures post propagation through an epidemic-style gossip protocol without internet connectivity.

1.3.2 Aim and Objectives

The aim of our project is to develop and deliver a serverless, peer-to-peer social media platform that operates in internet-deprived environments, ensuring the secure propagation of text and multimedia content without reliance on centralized

infrastructure.

The main objectives include:

- **Decentralizing Data Infrastructure**: Eliminating the need for central servers (such as Firebase) to ensure absolute user autonomy and censorship resistance.
- **Implementing P2P Content Propagation**: Developing a custom gossip protocol to facilitate the reliable broadcasting of text and images across a Bluetooth mesh network.
- **Ensuring Cryptographic Authenticity**: Utilizing the Ed25519 signature scheme to provide a verifiable and tamper-proof digital identity for every interaction within the network.
- **Optimizing Offline-First Persistence**: Maintaining local-first records using the Room Persistence Library to allow users to browse feeds and manage posts without active internet connectivity.
- **Multimedia Handling in Low-Bandwidth Environments**: Integrating automated image compression to enable the sharing of visual information across Bluetooth-constrained channels.

1.3.3 Scope

The application covers Bluetooth mesh networking, Ed25519 cryptographic signatures for post authenticity, and a local-first architecture using a Room database.

1.3.4 Definitions, Acronyms, and Abbreviations

- **P2P**: Peer-to-Peer
- **TTL**: Time-to-Live
- **Ed25519**: Elliptic Curve signature scheme used for authenticity
- **DAO**: Data Access Object

Chapter 2

Literature Review

2.1 Literature Survey

This chapter contains the literary survey conducted to support the decentralized architecture of Neo and the technologies used to build it. It includes a survey of existing communication challenges and a market analysis that justifies the necessity of an offline-first social platform from a privacy and resilience perspective.

2.1.1 The Shift Toward Decentralization

Traditional social media platforms reached a peak in global influence around 2010, but this growth led to increased concerns regarding data ownership and central censorship. As privacy breaches became more frequent, a global movement toward decentralized social networking emerged. To support this, research into peer-to-peer (P2P) protocols intensified, focusing on reducing reliance on central cloud servers that are vulnerable to government shutdowns and natural disasters.

2.1.2 Communication Issues in Internet-Deprived Regions

In countries like Pakistan, internet access is often inconsistent due to infrastructure limitations or intentional service outages. Centralized apps like WhatsApp or Facebook become unusable during these periods, leading to a total loss of community information flow. There is a dire need to solve this issue through local-first mesh networks that allow information to "hop" between devices without requiring a gateway to the global internet.

2.1.3 Cryptographic Authenticity and Ed25519

Digitization is a modern solution to trust issues in open networks. In a decentralized environment, the "patwari" or central authority is replaced by a cryptographic ledger. Neo utilizes the Ed25519 signature scheme, a modern Edwards-curve algorithm, to ensure that every post—whether text or image—is verifiable and tamper-proof. This proves instrumental in preventing corrupt practices such as identity spoofing and message manipulation in an open mesh.

2.1.4 Kotlin, Jetpack Compose, and Room

Upon researching different frameworks, it was noted that most traditional apps use a hybrid model involving Firebase for real-time synchronization. However, a major pivot was made in Neo's development on December 15, 2025, to remove Firebase and move toward a 100

- **Kotlin & Jetpack Compose**: These are suited for building modern, efficient, and responsive UIs with Material 3 standards.
- **Room Persistence Library**: This allows Neo to maintain "local-first" land-like records of posts and multimedia, ensuring data is never lost during disconnections.

2.2 Market Survey

Currently, there are no mainstream social media platforms in Pakistan that provide a fully serverless, P2P multimedia experience. Existing platforms related to communication and record management in the region include:

2.2.1 Bridgefy

A web-based mesh messaging app that gained popularity during protests and disasters. However, it lacks a persistent, structured social feed and verifiable cryptographic signatures for long-term data integrity.

2.2.2 Manyverse

An internet-less social network based on the Scuttlebutt protocol. While functional, it is not optimized for the specific low-bandwidth Bluetooth constraints and modern Android Material 3 aesthetics found in Neo.

2.2.3 Briar

A secure messaging app for activists and journalists. It focuses on 1-on-1 communication, whereas Neo is designed for wide-area broadcasting and public "land-record" style social feeds using a custom Gossip Protocol.

2.3 Conclusion of Survey

Keeping the future in mind, a Kotlin-based application utilizing a local-first Room DB was the best choice. It handles the three core needs of Neo: Persistence (storage), Security (Ed25519), and Propagation (Bluetooth Gossip) equally well.

Chapter 3

Software Requirement Specification

3.1 Functional Requirements

The functional requirements define the specific behaviors and services that the Neo application provides to its users and peers within the mesh network.

3.1.1 Multimedia Content Creation and Management

- **Post Creation**: The system shall allow users to create posts containing text, images, or a combination of both.
- **Image Compression**: The application must automatically process and compress images to a web-optimized format before transmission to ensure mesh stability.
- **Local Persistence**: The system shall store all created and received posts in a local Room database to allow for offline-first browsing.

3.1.2 Peer-to-Peer Communication

- **Neighbor Discovery**: The system shall continuously scan for and identify nearby peers using Bluetooth Classic and BLE.
- **Handshake and Sync**: Upon establishing a connection, the system must perform a handshake to exchange post summaries and synchronize missing content.
- **Epidemic Broadcasting**: The application shall implement a gossip protocol to forward verified posts to other connected peers, ensuring network-wide propagation.

3.1.3 Security and Identity

- **Key Generation**: The system shall generate a unique Ed25519 public/private key pair for each user upon initial setup to establish a verifiable identity.
- **Digital Signing**: The application must sign every outgoing post with the user's private key to ensure data authenticity.

- **Signature Verification**: The system shall verify the digital signature of every incoming packet against the sender's public key; if verification fails, the packet must be discarded.

3.1.4 User Interface and Monitoring

- **Social Feed Rendering**: The application shall display a chronological feed of all verified posts retrieved from the local database.
- **Mesh Status Monitoring**: The system shall provide a real-time status view showing the number of active peer connections and the current state of the Bluetooth hardware.

3.2 Non-functional Requirements

These requirements define the quality attributes and constraints of the system, focusing on how it should perform to ensure a reliable decentralized experience.

3.2.1 Performance and Scalability

- **Local Rendering Speed**: The user interface, built with Jetpack Compose, must render the local feed and load images from the Room database in under 500ms to maintain a fluid user experience.
- **Resource Optimization**: The application shall implement discovery throttling and periodic Bluetooth scanning to minimize battery consumption during the 227-day testing period.
- **Concurrent Peer Handling**: The system must maintain stable data synchronization with up to 7 concurrent Bluetooth peer connections without degrading the performance of the local UI.

3.2.2 Reliability and Mesh Availability

- **Eventual Consistency**: The Gossip Protocol must ensure that text and compressed image packets reach 95
- **Fault Tolerance**: The system must automatically recover from sudden Bluetooth connection drops by attempting background reconnections every 60 seconds.
- **Data Integrity**: The local storage system must ensure that no data corruption occurs during sudden power losses or application crashes by utilizing Room's transactional integrity.

3.2.3 Security and Privacy

- **Cryptographic Hardening**: Every interaction in the mesh must be secured by an Ed25519 signature, ensuring that invalid or tampered posts are rejected before they reach the local feed.

- **Local Encryption**: All sensitive metadata stored on the device must be protected using AES-256-GCM encryption at rest, with keys managed by the Android Keystore.
- **Anonymity**: The system shall not require a central phone number or email registration, maintaining user privacy as per the decentralized pivot made on December 15, 2025.

3.2.4 Usability and Accessibility

- **Connection Visibility**: The UI must provide real-time visual indicators showing the current Bluetooth status and the count of active peers in the mesh.
- **Offline Accessibility**: Users must be able to create posts and browse previously synced multimedia content even when the device is completely isolated from other peers.

3.3 System Features

3.3.1 Epidemic Gossip Propagation

The primary mechanism for data dissemination in Neo is the Gossip Protocol, which ensures that posts reach all nodes in the mesh without a central coordinator.

- **Broadcast Mechanism**: When a user creates a multimedia post, the system serializes the data and broadcasts it to all currently connected Bluetooth peers.
- **Hop Limit (TTL)**: Each post contains a Time-to-Live (TTL) value that decrements with every "hop" between devices to prevent infinite broadcast loops and network congestion.
- **Relay Logic**: Upon receiving a new post, a peer verifies the Ed25519 signature; if valid, the peer stores the post and automatically re-broadcasts it to its own unique set of connected peers.

3.3.2 Anti-Entropy Synchronization

To handle the dynamic nature of a mesh network where peers frequently join and leave, the SyncManager ensures eventual consistency.

- **Delta Exchange**: During a Bluetooth handshake, peers exchange a summary of their local post IDs to identify missing content.
- **Selective Retrieval**: The system only requests and transfers the missing "deltas" (posts created since the last successful sync), which optimizes the low bandwidth of Bluetooth Classic and BLE.
- **Conflict Resolution**: Since all posts are timestamped and cryptographically signed, the system automatically resolves conflicts by prioritizing the authentic author's original data.

3.3.3 Multimedia Processing and Storage

This feature manages the lifecycle of text and image data from creation to local persistence.

- **Automated Compression**: The system integrates an image processing pipeline that scales and compresses high-resolution photos into a mesh-friendly format before transmission.
- **Local Persistence Strategy**: Neo utilizes a "local-first" architecture where the Room database acts as the primary source of truth, allowing for instant feed rendering even in total isolation.

3.3.4 Cryptographic Identity and Verification

Security is integrated as a core system feature rather than an external layer.

- **Key Generation** Upon the first launch, the CryptoManager generates an Ed25519 key pair stored in the Android Keystore to establish a permanent, verifiable identity.
- **Seamless Verification**: The system performs background verification of all incoming packets, silently discarding any content that fails the signature check to maintain network trust.

3.4 Interface Requirements

3.4.1 User Interface (UI)

The user interface serves as the primary bridge between the user and the decentralized mesh network.

Design Framework: The UI shall be developed using Jetpack Compose and Material Design 3, ensuring a modern and responsive experience on Android devices (API 26+).

- **Aesthetic Standards**: The application must support a "dark-mode-first" aesthetic with dynamic color schemes that adapt to system-level preferences.
- **Multimedia Rendering**: The feed interface must include an Enhanced-PostCard component capable of rendering text alongside locally stored images.
- **Status Indicators**: The interface shall provide real-time visual feedback, including Bluetooth toggle status, a "peers discovery" progress bar, and a count of active connections.

3.4.2 Communication Interface

This requirement defines the low-level protocols used for device-to-device interaction.

- **Protocol Selection**: The system shall interface with the Android Bluetooth stack to establish peer-to-peer (P2P) sockets using the RFCOMM (SPP) protocol.
- **Connection Management**: The PeerConnection class must manage raw byte streams to facilitate the transfer of serialized JSON data and compressed image bytes.
- **Hardware Compatibility**: The system shall support communication via both Bluetooth Classic (for higher bandwidth image transfer) and BLE (for energy-efficient neighbor discovery).

3.4.3 Software and Hardware Interfaces

- **Database Interface**: The application logic shall interact with the Room Persistence Library via Data Access Objects (DAOs) to perform CRUD operations on text and image metadata.
- **Security Interface**: The CryptoManager must interface with the Android Keystore System to perform hardware-backed Ed25519 signing and verification.
- **Hardware Access**:The application must request and handle Android system permissions for BLUETOOTH CONNECT, BLUETOOTH SCAN,and ACCESS FINE LOCATION to operate the mesh hardware.

3.5 Glossary

3.5.1 Glossary

The following terms are used throughout this report to describe the technical architecture and operations of the Neo platform:

- **BLE (Bluetooth Low Energy)**: A power-conserving variant of Bluetooth designed for periodic communication between devices.
- **Decentralization**: A system architecture where no central server or authority (like Firebase) controls data; all nodes have equal functional capability.
- **Ed25519**: A modern Edwards-curve Digital Signature Algorithm (EdDSA) used in Neo to provide fast and secure post authentication.
- **Gossip Protocol**: A peer-to-peer communication pattern where nodes periodically share information with their neighbors to ensure network-wide data propagation.
- **Mesh Network**: A network topology where each node (device) relays data for the network, allowing information to "hop" between peers to reach distant nodes.

- **P2P (Peer-to-Peer)**: A distributed application architecture that partitions tasks or workloads between peers without the need for central coordination.
- **Room Persistence**: A local-first Android database library that provides an abstraction layer over SQLite for robust data storage.
- **TTL (Time-to-Live)**: A mechanism that limits the lifespan or number of hops a post can take within the mesh to prevent infinite loops.

3.6 Scope of Project

The scope of the Neo project defines the operational boundaries and the technical depth of the decentralized social media platform developed during this 227-day period.

3.6.1 In-Scope Features

The project includes the design, development, and testing of the following core functionalities:

- **Decentralized Architecture**: Development of a 100% serverless environment that functions without Firebase or central databases, as finalized in December 2025.
- **Mesh Networking**: Implementation of an epidemic-style gossip protocol using Bluetooth Classic/BLE for peer-to-peer data propagation.
- **Cryptographic Security**: Integration of Ed25519 signatures to ensure message authenticity and non-repudiation in an open mesh network.
- **Local Persistence**: A local-first data strategy utilizing the Room Persistence Library for offline feed browsing and storage.
- **User Interface**: A modern Android UI built with Jetpack Compose following Material Design 3 standards.
- **Multimedia Support**: Users can create posts containing text and/or images, which are stored locally and shared with peers via the gossip protocol.
- **Image Compression**: Implementation of an image processing layer to compress files before transmission to maintain mesh stability.

3.6.2 Out-of-Scope (Limitations)

To maintain project feasibility within the academic timeline, the following areas are excluded:

- **High-Definition Video**: While text and images are supported, high-definition video streaming is excluded to prevent network congestion.

- **Global Internet Bridge**: While synchronization via internet-connected devices was initially explored (Meeting Log 02), the final version focuses purely on offline local mesh connectivity[cite: 46, 65].
- **iOS Compatibility**: The application is developed exclusively for the Android ecosystem (API 26+).

Chapter 4

Proposed Solution

4.1 Project Features and Functionalities Overview

4.1.1 User Interface

The UI is developed using Jetpack Compose with a focus on a "Dark Mode" aesthetic. It features a Material 3 design system, including an EnhancedPostCard for feed items and an intuitive NavigationBar for seamless transitions between the Feed, Profile, and Mesh Status screens.

4.1.2 Translation Process (Data Serialization)

In Neo's decentralized context, the "translation" process refers to the serialization of Kotlin data models into JSON format using the Gson library. This allows complex Post and Device objects to be converted into byte streams for transmission over Bluetooth RFCOMM sockets.

4.1.3 Error Handling

The system implements robust error handling for Bluetooth connection drops, including automatic reconnection logic in the PeerConnection class and cryptographic verification failures where invalidly signed posts are silently rejected to prevent spam.

4.1.4 User Management

Users are managed locally through a unique Device UUID and a corresponding Ed25519 public/private key pair generated upon the first application launch. No central registry is used; peer identities are established during the Bluetooth Handshake phase.

4.1.5 Translation History (Sync Logic)

The SyncManager maintains a history of the last synchronization timestamp for each peer. This ensures that when two devices reconnect, they only exchange posts created since their last successful interaction, optimizing bandwidth.

4.1.6 Performance and Usability

The app ensures low latency by utilizing a Room database for local caching, allowing the UI to remain responsive even during heavy background mesh synchronization. Usability is enhanced through real-time connection status indicators.

4.1.7 Security and Reliability

Reliability is achieved through the Gossip Protocol, which ensures multiple propagation paths. Security is grounded in Ed25519 signatures, ensuring message non-repudiation and integrity.

4.1.8 Scalability and Maintainability

The project uses Hilt for Dependency Injection and Clean Architecture, making the codebase highly modular. The mesh architecture scales naturally as more nodes increase the network's density and reach.

4.1.9 Compatibility and Accessibility

Neo supports Android devices from API 26 (Android 8.0) up to API 34, ensuring compatibility with a wide range of hardware.

4.2 Detailed System Architecture

The Neo system is built using a Clean Architecture approach to separate concerns and ensure the codebase remains maintainable and testable over the 227-day development lifecycle. The architecture is divided into three primary layers that communicate without a central authority.

4.2.1 Presentation Layer (User Interface)

- **Jetpack Compose**: The modern UI layer handles user interactions, rendering the social feed and image previews using Material Design 3 components.
- **ViewModels**: These act as the bridge between the UI and the underlying logic, managing state and ensuring that mesh updates (like new peer connections) are reflected in real-time.

4.2.2 Domain and Logic Layer

- **Gossip Protocol & SyncManager**: This core module implements the epidemic broadcasting logic; it manages the Time-to-Live (TTL) of posts and handles the "delta exchange" to synchronize missing text and image data.
- **CryptoManager**: Responsible for the security of the mesh, this module handles Ed25519 signature generation and verification to ensure non-repudiation of every post.

- **Image Processing Pipeline**: Before any image is sent to the mesh, this module compresses and resizes the file to ensure it meets the bandwidth constraints of Bluetooth.

4.2.3 Data and Hardware Layer

- **Room Persistence Library**: Acts as the local-first "Source of Truth," storing post metadata, text content, and local file paths for images.
- **Bluetooth RFCOMM Service**: This hardware-facing module manages the low-level socket streams, facilitating the actual peer-to-peer byte transfers between Android devices.
- **Android Keystore**: Provides hardware-backed storage for the Ed25519 private keys, ensuring they are never exposed to other applications or the network.

4.3 Model

The core logic follows an offline-first **Post Model**. Each post is a cryptographically signed object containing a unique ID, author metadata, and a Time-to-Live (TTL) counter to prevent infinite broadcast loops within the mesh.

4.3.1 Development Methodology (Agile Model)

The project utilizes the **Agile Development Model**, specifically an iterative sprint-based approach. This methodology focuses on delivering a functional product through continuous cycles of planning, design, coding, and testing. Neo was developed using the Agile model, focusing on an iterative delivery process that allowed the team to adapt to the complex challenges of Bluetooth mesh networking and decentralized data integrity. Phases of the Agile Model for Neo:

- **Concept**: The initial scope focused on establishing a basic peer-to-peer messaging system. We determined that a decentralized social platform was feasible using Android's Bluetooth stack without requiring a central server.
- **Inception**: We created a Work Breakdown Structure (WBS) for a 227-day timeline. Initial designs included mockups for the Jetpack Compose UI and a schema for the Room database to handle local-first persistence.
- **aIteration**: Development was divided into four primary sprints. A major pivot occurred in the second sprint on December 15, 2025, when we decided to remove Firebase to ensure a 100
- **Release**: The product was tested using multi-device mesh simulations to verify Bluetooth socket stability and the gossip protocol's reliability. We utilized Android's internal testing tools to ensure cryptographic signature verification worked across all nodes.

- **Maintenance**: Post-release, the team will support the application for one year, focusing on optimizing the Gossip Protocol's battery consumption and refining the UI based on user feedback.
- **Retirement**: The software lifecycle will end with a final data export feature, allowing users to move their local Room records to a successor platform if the system is ever discontinued.

4.4 UML Design Diagrams

The following UML diagrams represent the structural and behavioral aspects of the Neo platform, ensuring a clear understanding of its decentralized operations.

4.4.1 Use Case Diagram

The Use Case diagram illustrates the interactions between the primary actors (User and Peer) and the decentralized system.

- **Actor - User**: The local human operator who creates content, attaches images, and monitors the network state.
- **Actor - Peer**: A remote Android device that acts as a node for relaying data and facilitating the mesh structure.
- **Core Use Cases**: These include Broadcast Post (Epidemic Gossip), Verify Signature (Ed25519), Sync Feed (Anti-entropy synchronization), and Image Compression (Media handling).

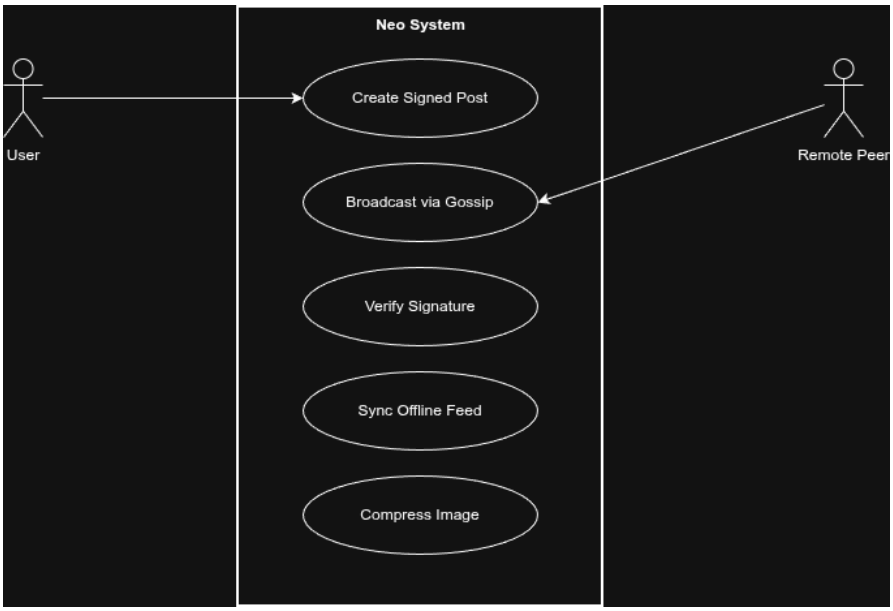

Figure 4.1: Neo Use Case Diagram

4.4.2 Class Diagram

The Class Diagram displays the system's static structure and Clean Architecture layers.

- **Bluetooth Layer**: Contains 'BluetoothService' and 'PeerConnection' classes to manage raw socket streams.
- **Sync Layer**: Comprises 'GossipProtocol' and 'SyncManager' to handle message routing logic.
- **Data Layer**: Includes 'PostDao' and 'PostRepository' for managing local Room database persistence.
- **Security Layer**: Centered around 'CryptoManager' for Ed25519 signing and verification.

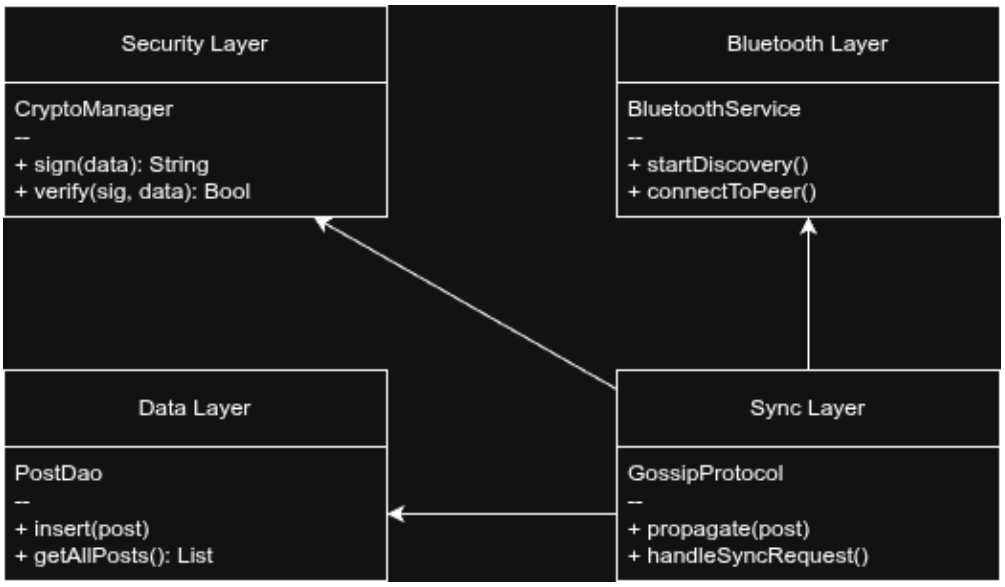

Figure 4.2: Neo Class Diagram showing Architecture Layers

4.4.3 Activity Diagram

This diagram maps the internal logic when the system receives a data packet from the mesh.

- **Input**: A raw byte stream is received via the Bluetooth RFCOMM socket.
- **Verification**: The system uses the CryptoManager to validate the Ed25519 signature before any processing.
- **Decision Node**: If the signature is invalid, the packet is discarded; if valid, it checks for image data and initiates the Room storage process.

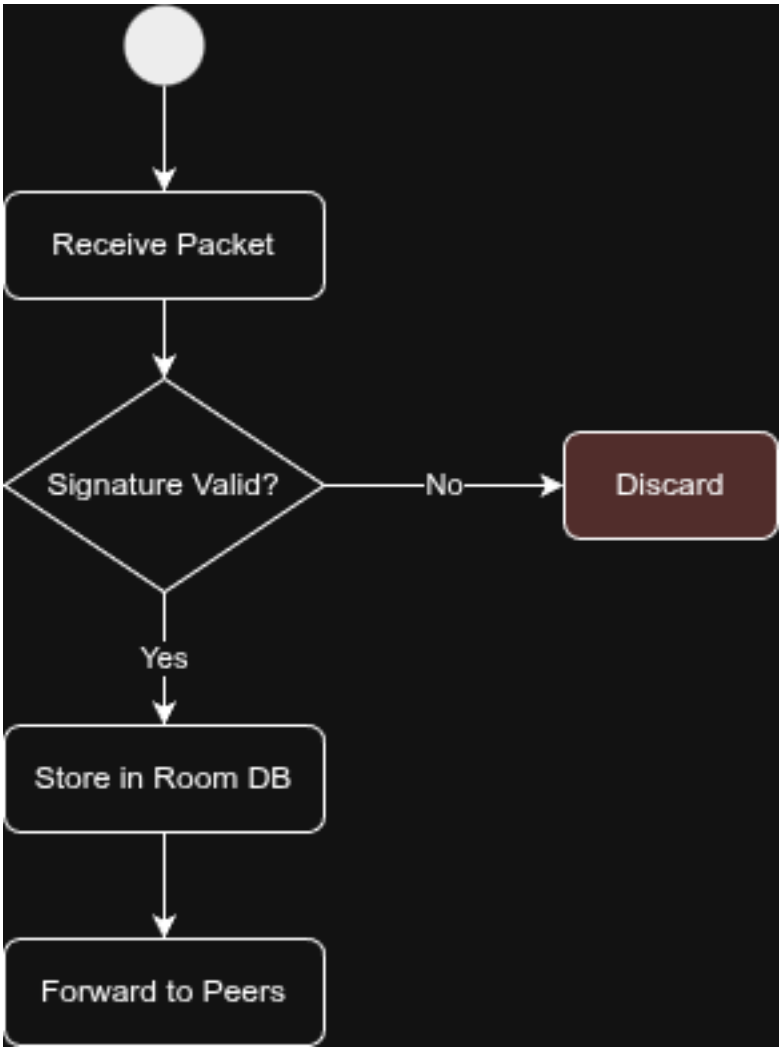

Figure 4.3: Neo Activity Diagram (Packet Verification)

4.5 Sequence Diagram(Post Broadcast Lifecycle)

The sequence diagram details the time-ordered interactions between components during a post-creation event.

- **UI to ViewModel**: The user submits text and an image.
- **ViewModel to CryptoManager**: The payload is hashed and signed with the private key.
- **Protocol to Hardware**: The signed post is handed to the GossipProtocol, which sends it to the BluetoothService for multi-peer broadcasting.

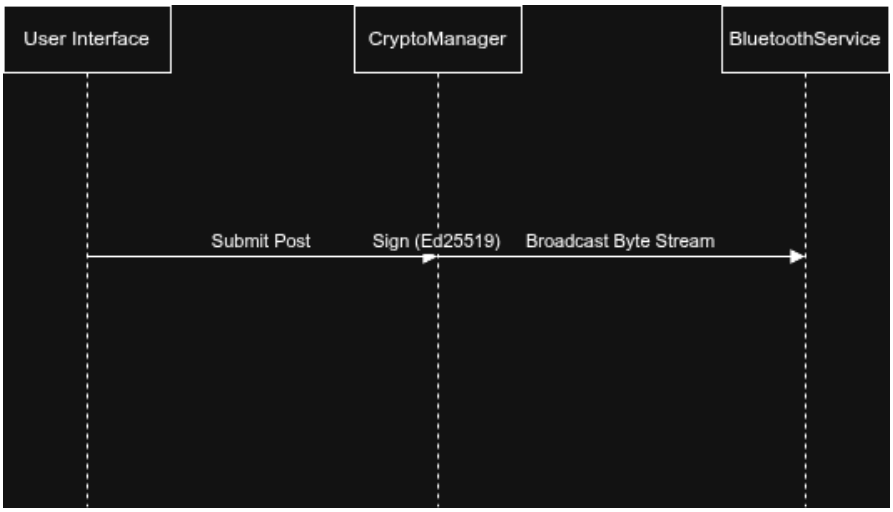

Figure 4.4: Neo Sequence Diagram

4.6 Component & Deployment Architecture

This diagram visualizes the logical components of the Clean Architecture and their physical distribution across Android hardware.

- **Logical Components**: Separates the Jetpack Compose UI, Room Database, and Bluetooth Networking modules.
- **Deployment Node**: Represents an Android Device (API 26+) utilizing the Android Keystore for hardware-backed security.
- **P2P Link**: Defines the wireless connection between devices via Bluetooth RFCOMM.

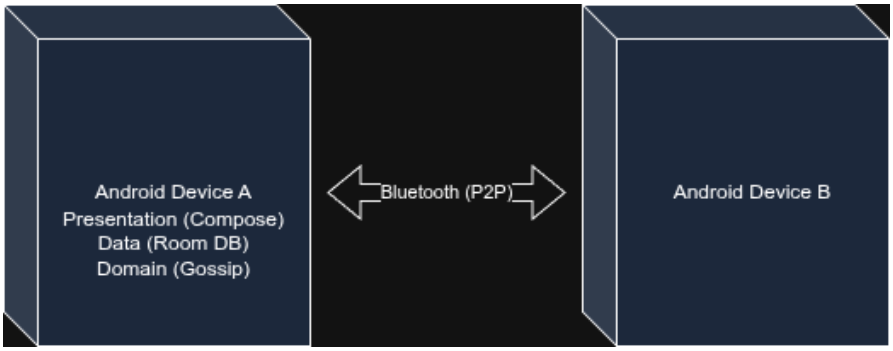

Figure 4.5: Neo Deployment Architecture

4.7 Database Design (Room Schema)

The database layer of Neo is designed with a local-first approach, ensuring that all interactions are persisted on the device before and after mesh propagation.

4.7.1 Entity Relationship Details

- **Post Entity**: This is the primary table representing the social feed. It contains the unique ID of the post, the author's metadata, and the raw text content.
- **Media Entity**: To optimize storage, image data is handled separately. This table stores the local URI of the compressed image, its cryptographic hash, and a foreign key link to the parent Post Entity.
- **Peer Entity**: This table tracks the "handshake history" with other nodes. It stores the remote device's UUID, its Ed25519 public key, and the timestamp of the last successful synchronization.

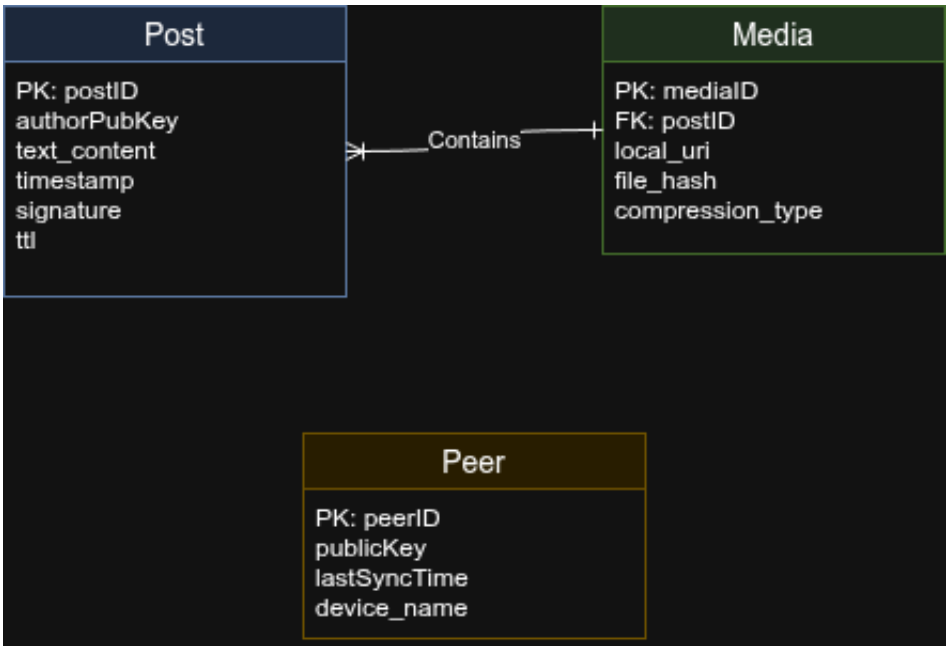

Figure 4.6: Neo Entity-Relationship Diagram (Room Database Schema)

4.7.2 Flow Diagram (Content Lifecycle)

The Flow Diagram (also referred to as a Data Flow Diagram) illustrates how a multimedia post moves through the system's internal layers during a broadcast event.

- **Input Layer**: Captures user input (text/image) and passes it to the Image Compressor.
- **Security Layer**: The CryptoManager hashes the combined payload and generates a digital signature.

- **Persistence Layer**: The signed data is saved to the Room Database as the local source of truth.
- **Transmission Layer**: The Gossip Protocol wraps the signed post into a JSON packet and broadcasts it via the Bluetooth RFCOMM socket.

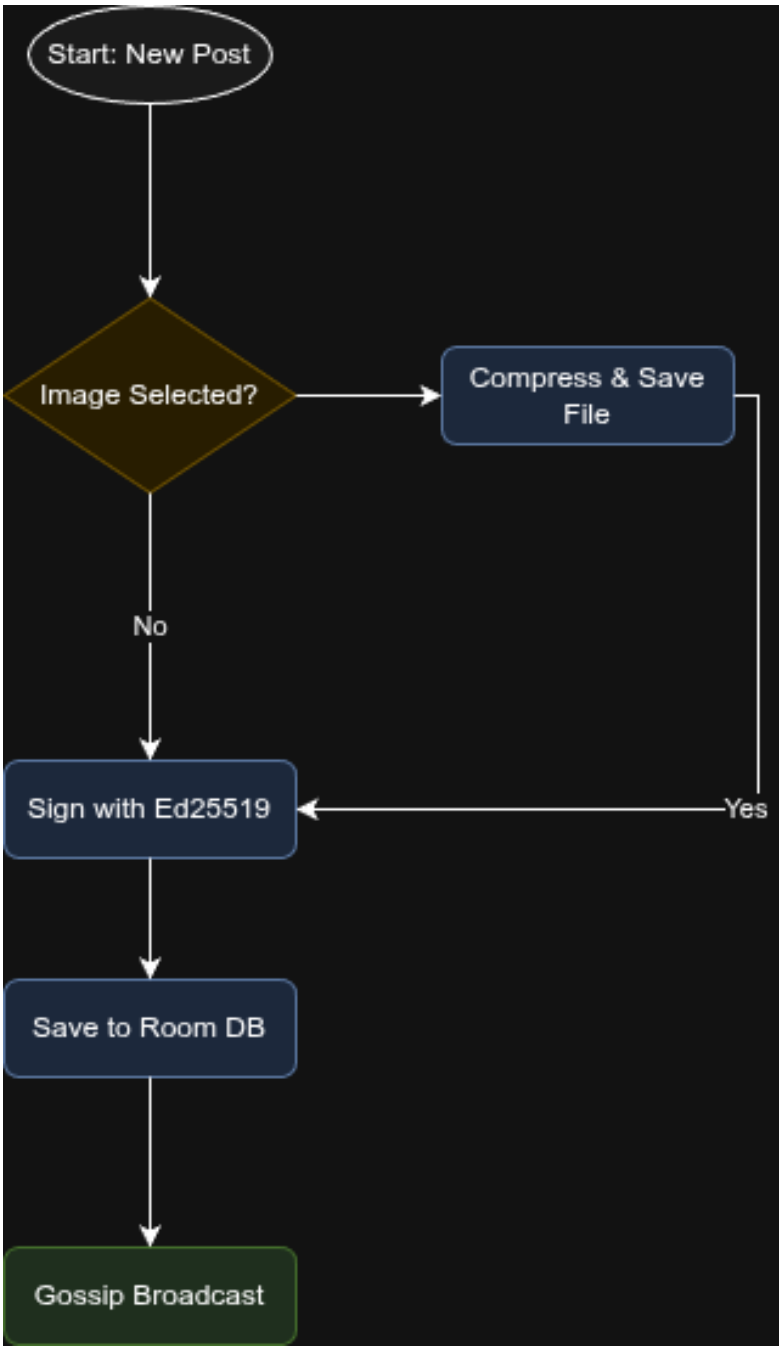

Figure 4.7: Neo Content Lifecycle Flow

4.8 Experimental / Simulation Setup

The setup for Neo involved specific hardware and software to facilitate decentralized P2P testing.

- **Hardware**:Development Laptops running Zorin OS for coding and documentation. A minimum of three Android devices (API 26+) to facilitate multi-node mesh testing and verify message propagation.
- **Software**:Android Studio: Used as the primary IDE for Kotlin and Jetpack Compose development.
- **Room Persistence Library**: For local-first database management and offline data handling.
- **Ed25519 Libraries**: For implementing cryptographic signing and verification.
- **StarUML**: For designing the Use Case, Class, and Sequence diagrams.

4.9 Milestones Achieved

The following work packages have been completed for Neo:

- **Architectural Pivot**: Successfully transitioned from a hybrid model to a 100
- **Gossip Protocol Implementation**: Tested and verified epidemic-style data propagation across a 3-node mesh.
- **Multimedia Integration**: Developed and integrated an image compression pipeline for Bluetooth-friendly multimedia sharing.
- **Security Layer**: Integrated hardware-backed Ed25519 signatures via the Android Keystore for user authenticity.

4.10 Evaluation Parameters

Neo is evaluated based on the following quality and performance metrics:

- **Peer-to-Peer Reliability**: Successful propagation of text and images to 95
- **Signature Integrity**: A 100
- **UI Performance**: Local feed rendering and database queries must be completed in under 500ms to ensure a smooth experience.
- **Decentralization Verification**: Verification that full application functionality remains intact while the device is in total isolation from the internet.

4.11 Project Timeline (Gantt Chart)

The following Gantt chart outlines the 184-day schedule from initial concept to final deployment.

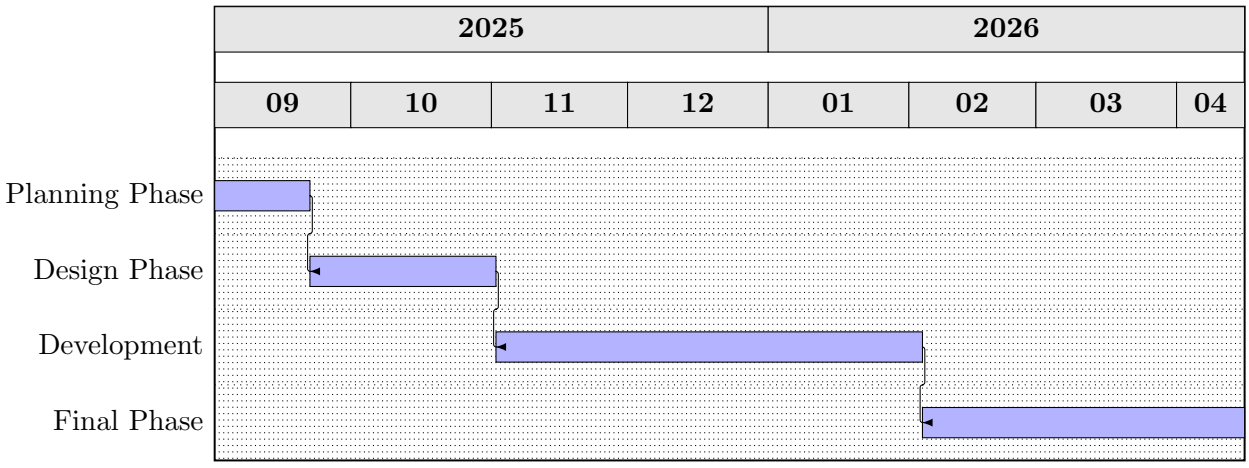

Figure 4.8: Neo Project Gantt Chart

4.12 Detailed Work Plan

The Neo project was executed over a span of 227 days using an Agile iterative approach. The breakdown of tasks and their specific timelines, aligned with supervisor feedback, are as follows:

- **Planning Phase (21 Days) [2025-09-01 to 2025-09-21]**:
  - Initial requirements gathering and feasibility study.
  - Decided on the core feature set and technical stack (BLE + Initial Firebase research)[cite: 13].
- **Design Phase (41 Days) [2025-09-22 to 2025-11-01]**:
  - UI/UX design and flow approval for primary screens[cite: 21].
  - Finalized feature list and explored internet-connected device synchronization[cite: 18, 20].
- **Development Phase (94 Days) [2025-11-02 to 2026-02-03]**:
  - **Major Pivot**: Removed Firebase integration to ensure the system remains 100% decentralized and serverless.
  - Shifted focus entirely to BLE mesh-based data sharing and offline consistency.
  - Implemented core feed and posting features with UI performance tuning.

- **Final Phase (71 Days) [2026-02-04 to 2026-04-15]**:
  - Extended integration testing across multiple nodes to handle data consistency without a central server.
  - Final UI refinements and technical report compilation.

4.13 Scalability and Maintainability

The mesh architecture allows the network to scale naturally with the number of participating devices, while the Hilt dependency injection framework ensures codebase maintainability.

Chapter 5

Conclusion & Analysis

5.1 Summary of Findings

The following table evaluates the project's success in meeting the core objectives established at the beginning of the development lifecycle.

Objective	Implementation Feature	Status
Decentralization	100% Serverless (Removed Firebase on Dec 15, 2025).	Achieved
P2P Propagation	Epidemic Gossip Protocol via Bluetooth.	Achieved
Authenticity	Ed25519 Cryptographic Signatures.	Achieved
Offline Persistence	Local-first Room Database Architecture.	Achieved
Multimedia	Automated Image Compression for Mesh Stability.	Achieved

Table 5.1: Summary of Project Objective Fulfillment

5.2 Project Benefits

The implementation of Neo as a fully decentralized social media platform provides several critical benefits for modern digital communication:

- **Censorship Resistance**: By removing the central server (Firebase), as decided during the project review on December 15, 2025, the platform ensures that no single entity can shut down or control the flow of information.
- **Cost Efficiency**: Since the project relies on peer-to-peer (P2P) Bluetooth mesh networking rather than cloud infrastructure, the operational costs for both developers and users are effectively zero.
- **Resilience in Internet-Deprived Regions**: Neo facilitates communication in areas with poor infrastructure, during natural disasters, or in regions where internet access is restricted.

- **Privacy and Security**: The use of Ed25519 cryptographic signatures ensures that all posts are authentic and non-repudiable without requiring a centralized identity provider.

5.3 Conclusion

The development of Neo successfully demonstrates the feasibility of a fully decentralized, serverless social media platform for Android devices. Over the course of the 227-day project lifecycle, the team successfully transitioned from a hybrid infrastructure to a 100

By utilizing the Gossip Protocol over Bluetooth Classic and BLE, Neo achieves resilient data propagation even in environments with zero internet connectivity. The integration of Ed25519 cryptographic signatures provides a robust security layer, ensuring that every text and image post is verifiable and tamper-proof. Furthermore, the application of modern development tools like Jetpack Compose and Room Persistence ensures a high-performance, responsive user experience that aligns with current industry standards. In conclusion, Neo serves as a critical proof-of-concept for secure, censorship-resistant communication in emergency, remote, or privacy-sensitive scenarios.

5.4 Recommendations

For future iterations and scalability of the Neo platform, the following recommendations are provided:

- **Fragmented Data Transmission**: To improve image sharing performance, we recommend breaking large images into smaller packets to be reassembled by the receiving peer, reducing Bluetooth socket timeouts.
- **Protocol Optimization**: Further research into "Scuttlebutt" or similar anti-entropy gossip protocols could reduce redundant data transmissions and save battery life across the mesh.
- **Hardware Expansion**: Integrating Wi-Fi Direct alongside Bluetooth would significantly increase the data transfer rates for media-heavy posts and larger file types.
- **Enhanced Encryption**: Implementing end-to-peer encryption (E2EE) for private messaging layers would further secure user communications from passive mesh listeners.

Bibliography

[1] Android Developers. "Bluetooth overview," *Android Developers Documentation*. [Online]. Available: https://developer.android.com/guide/topics/connectivity/bluetooth

[2] Android Developers. "Save data in a local database using Room," *Android Developers Documentation*. [Online]. Available: https://developer.android.com/training/data-storage/room

[3] Jetpack Compose. "Build better apps faster with Jetpack Compose," *Android Developers Documentation*. [Online]. Available: https://developer.android.com/jetpack/compose

[4] D. J. Bernstein. "Ed25519: high-speed high-security signatures," *Ed25519.cr.yp.to*. [Online]. Available: https://ed25519.cr.yp.to/

[5] W. Ali, H. Raza, and M. Zaka. "Neo: A Decentralized Social Media Platform - Project Meeting Logs," National Skills University Islamabad, Dec. 2025.

[6] A. S. Tanenbaum and M. Van Steen. *Distributed Systems: Principles and Paradigms*. Pearson Education, 2007.